

AAI500 - Applied Artificial Intelligence

San Diego University

Customer Credit Default Prediction

Submitted to: Instructor Azka Azka

Submitted by:

Anugrah Rastogi

Mallesham D

Dhrub Satyam

Table of Content

[Table of Content](#)

[Abstract](#)

[1. Introduction](#)

[2. Data Processing](#)

[3. Data Exploration and Insights](#)

[Feature Selection](#)

[4. Model Selection](#)

[5. Model Analysis](#)

[6. Conclusion and Recommendations](#)

[Appendix \(Code and Output\)](#)

[References](#)

Abstract

This document presents a data-driven approach to identifying customers who may fail to repay their credit obligations. The study uses structured financial records that include behavioral and personal characteristics to build and evaluate prediction tools.

After cleaning the data for missing values, converting categories, and scaling numeric fields, we scaled it for pattern and distribution analysis. We trained and validated the XGBoost, Random Forest, Decision Tree, and Logistic Regression algorithms.

To determine performance of each model, we used the metrics of F1 Score, Precision Score, ROC AUC and Recall Score. Among all the tested algorithms, the predictions of the Random Forest algorithm are very reliable due to accurate interpretability and accurate prediction.

1. Introduction

Figuring out if a customer might miss a credit payment is a real-world challenge that banks and financial companies face every day. The ability to identify risky borrowers before issuing credit can significantly reduce financial losses, streamline approval workflows, and enhance regulatory compliance.

This project, undertaken as part of the AAI500 course at the University of San Diego, leverages machine learning techniques on a real dataset containing both behavioral and demographic factors. This analysis is based on a public dataset available from an online platform (Kaggle, 2023).

Our objective is to build a model that can find out whether the customer will default or not on his/her credit. Apart from it, another objective of the project was to find out which of the ML techniques is good in terms of accuracy and practicality. Finally, this project will help gain insights that could make credit decision-making easier.

2. Data Processing

To provide consistency within the dataset (Kaggle, 2023), the median of each numeric type column is used to fill the missing values. The system ensures use of unskewed or original data for analysis. This helped eliminate the risk of bias due to outliers.

Dataset Overview:

- The Training Set has 70,000 records with 82 features.
- Test Set: 33,000 records with 80 features
- Target Column: The target column indicates whether a customer has defaulted (1) or not (0).

Data Preparation and Transformation:

- **Managing Missing Data:** Missing values in numerical features were imputed using their respective median values. The median was preferred because it is robust to outliers, ensuring that extreme values do not skew the data.
 - Example: If the income column had missing entries, they were filled with the median income value of the entire column.
- **Categorical Encoding:** To handle categorical data, one hot encoding was applied, transforming string or Boolean values into numeric form. Encoding was applied consistently across both training and testing data. To maintain uniformity, the encoder was fit using the combined set of unique category values from both datasets.
 - Example: Gender: {'Male' → 0, 'Female' → 1}
- **Feature Scaling:** We used RobustScaler to scale numerical features. Unlike standard scaling methods, RobustScaler uses the interquartile range (IQR), which makes it highly effective in the presence of outliers and models are not overly influenced by extreme values.
- **Feature Alignment:** To prevent inconsistencies during prediction, we made sure the training and test datasets had the same columns. This included, Dropping extra columns if present in either set and Ensuring consistent column order.
- **Target Isolation:** We labeled the target column such that 0 corresponds to non-defaulters, while 1 marks cases where the customer defaulted.

3. Data Exploration and Insights

EDA provides an initial overview, helping us explore trends, anomalies, and relationships within the data. It involves spotting irregularities, exploring feature behavior, and forming early hypotheses. In this project, EDA allowed us to examine how different variables interact with the outcome (credit default), which was essential in selecting meaningful features for modeling.

Univariate Analysis

We began by examining distributions of key features to understand their spread, outliers, and potential impact.

- **Numerical Features:** The dataset consists of fields such as Number of open credit lines, Monthly Income, Age, and Debit Ratio. Some variables showed right-skewed distributions, especially financial ones like `debt_ratio` and `monthly_income`.
- **Categorical Features:** To prepare categorical variables of Boolean and object types for modeling, one-hot encoding is used to convert into numerical type. Certain categorical features had imbalanced value counts, which might influence the model bias if not handled properly.
- **Example Chart:** It revealed a long tail indicating a small number of customers with very high debt ratios.

Bivariate Analysis

We analyzed features related to the target variable.

- **Box Plots:** Here, `debt_ratio` and `number_of_dependents` were higher on average among defaulters.
- **Correlation Matrix:** Heatmaps helped spot weak to moderate correlations. Most features showed low linear correlation, suggesting nonlinear models (e.g., trees) might perform better.
- **Default Distribution:** Only ~20.6% of records were labeled as defaulters. This class imbalance required attention during model training (e.g., via `class_weight='balanced'` or resampling techniques).

Key Visual Insights

- **Target Imbalance:** Defaulted (1): ~20% and Non-defaulted (0): ~80%. This significant imbalance influenced our choice of metrics (F1-score, ROC AUC instead of accuracy).
- **Outliers:** Features like `monthly_income` had extreme values. Instead of dropping, we used `RobustScaler` to mitigate their influence.

- **Missing Data:** We found missing data in income and dependent count fields. Imputed using median values, preserving the distribution shape.

Feature Selection

The initial dataset contained 189 features. To enhance model performance, generalization, and minimize overfitting, we implemented a two-step feature selection strategy using statistical techniques.

Information Value (IV)

A metric used for classification is Information Value which tells us the ability of a feature to differentiate between two classes in the target variable. Experts use it to judge how much each variable is able to predict credit ratings. Features that have very low Information Value (IV) are not useful while very high IVs may indicate leakage.

IV Score Range	Interpretation
< 0.02	Not predictive
0.02 – 0.1	Weak predictive power
0.1 – 0.3	Moderate predictive power
0.3 – 0.5	Strong predictive power
> 0.5	Potential leakage

By applying IV filtering, we removed 85 features that were either insignificant or flagged as suspicious due to unusually high IV scores.

Pearson Correlation

To further eliminate redundancy, we calculated the correlation coefficient between all numerical features. Features with high pairwise correlation ($\geq 75\%$) can cause multicollinearity, leading to model instability and inflated variance.

This step led to the removal of 16 additional features that showed high correlation with other predictors.

Final Feature Set

After completing both IV and correlation-based filtering, we reduced the dataset from 189 to 72 features. This streamlined feature space improved both the efficiency and reliability of our machine learning algorithms.

Summary of EDA Outcomes

Insight	Action Taken
Class imbalance in target	Used class_weight='balanced' and chose suitable metrics
Skewed financial data	Applied RobustScaler
Categorical encoding needed	Used One Hot Encoder
Correlations were weak	Preferred ensemble models like Random Forest/XGBoost
Missing values in key columns	Used median imputation

4. Model Selection

Selecting an appropriate model is vital for any real-world, real-time data. This is particularly true in the case of imbalanced classes. The final set of 72 features was chosen to train the model using IV and correlation - based filtering. We conducted our selection across algorithms of different complexity and interpretability.

Candidate Models

We explored 4 supervised classification algorithms, each has its unique advantages.

Model	Rationale
Logistic Regression	Simple, interpretable, works well as a baseline.
Decision Tree	It is easy to visualize and explain decisions.

Random Forest	It handles non-linearities, robust to noise, good for tabular data.
XGBoost	It is known for its high accuracy and predicting power.

Hyperparameter Tuning

All models are optimized for ROC AUC through GridSearchCV, applying a 5-fold cross-validation strategy.

Logistic Regression:

- Logistic regression is the first go-to classification method, when the relationship between predictors and outcome is almost linear (Hosmer, Lemeshow, & Sturdivant, 2013).
- Grid: C values and solvers
- To correct for label disparity in the target variable, we enabled class balancing through the class_weight setting.

Decision Tree:

- Decision trees provide an intuitive structure for modeling decisions and are widely used due to their interpretability and low computational cost (Quinlan, 1986).
- To improve performance, we explored combinations of minimum_samples_leaf, maximum_depth, and minimum_samples_split during hyperparameter optimization.

Random Forest:

- Random Forest enhances decision tree performance by using ensemble learning to decrease overfitting and increase accuracy (Breiman, 2001).
- Hyperparameter optimization included adjusting maximum depth, estimator quantity, and the minimum split threshold.

XGBoost:

- XGBoost is a widely used library with high performance for boosting. It can handle sparse data efficiently and incorporates regularization to prevent overfitting (Chen and Guestrin, 2016).

- We did parameter tuning by max_depth, learning_rate, and number of estimators for better performance. The evaluation metric used during training was Log loss to assess classification quality.

Tools Used:

- GridSearchCV is used for tuning (Pedregosa et al., 2011a).
- RobustScaler is used to scale numerical features (Pedregosa et al., 2011b).
- One-hot encoding is used for categorical encoding (Pedregosa et al., 2011c).

Evaluation Strategy

To evaluate models fairly, we split the training data:

- **80% for training, 20% for validation**
- Performance was measured using:
 - **ROC AUC Score** (discrimination ability)
 - **F1 Score** (balance between precision & recall scores)
 - **Precision Score & Recall Score** (for defaulters)

Validation Results:

Model	ROC AUC	F1 Score	Precision	Recall
Decision Tree	0.8808	0.6855	0.75	0.63
Logistic Regression	0.9057	0.7134	0.63	0.82
XGBoost	0.9576	0.7906	0.82	0.76
Random Forest	0.9633	0.8049	0.85	0.76

Final Model Selection

The final model was selected visualizing F1 score and ROC AUC score. Outstanding ROC AUC score is achieved by Random Forest. This model also achieved strong precision, recall, and F1 scores, making it a suitable choice as the best final model for our analysis.

5. Model Analysis

After training and tuning, we evaluated the results on the validation set of each model to test its usefulness. We want to see which model can well identify a defaulter without making too many unnecessary false positives.

Classification Reports

The parameters precision, recall, and F1 metrics helped to assess how well selected algorithms detected defaulting customers, with a primary focus on class 1 instances.

Model	Precision (class 1)	Recall (class 1)	F1 Score (class 1)	ROC AUC
Logistic Regression	0.63	0.82	0.7134	0.9057
Decision Tree	0.75	0.63	0.6855	0.8808
Random Forest	0.85	0.76	0.8049	0.9633
XGBoost	0.82	0.76	0.7906	0.9576

Understanding the Evaluation Metrics:

- **Precision** tells us how accurate the model's default predictions were — in other words, Out of the predicted defaulters, how many were actually correct?.
- **Recall** measures how effectively the model detects customers who actually defaulted.
- **F1 Score** is a balanced metric of recall and precision scores, helping us understand overall model effectiveness for the default class.
- **ROC AUC** has the ability to correctly separate defaulters from non-defaulters across all thresholds — the higher, the better.

Random Forest led with the highest F1 and ROC AUC, making it best suited for production.

Visual Insights:

We plotted key evaluation curves to support our findings.

1. The ROC Curve indicates that the highest AUC is obtained in the case of Random Forest which is indicative of best classification.
2. The precision-recall curve shows that the model handles class imbalance well.
3. Model Comparison Bar Chart.

Model	ROC AUC	F1 Score
Random Forest	0.9633	0.8049
XGBoost	0.9576	0.7906
Logistic Regression	0.9057	0.7134
Decision Tree	0.8808	0.6855

Summary of Observations

- Random Forest performed best overall, especially in identifying defaults accurately.
- XGBoost was a close second and could be explored further for optimization.
- Logistic Regression, while simple, showed surprisingly good recall but low precision—making it risky for production.
- The Decision Tree was interpretable but too simplistic for this complex dataset.

Why Choose Random Forest?

- Among all the models we tested, we achieved the highest ROC AUC and F1-score.
- It demonstrates consistent performance across both defaulter and non-defaulter classes.
- It effectively handles noisy data and is resilient to outliers.
- It is recommended for datasets which contain a combination of numerical and categorical features, making it ideal for financial applications.

6. Conclusion and Recommendations

Project Recap:

This project focused on developing a machine learning technique to identify the likelihood of customer credit default using structured, real-world data. Dimensionality reduction via IV and correlation analysis helped in improving model efficiency and reducing noise, ultimately boosting performance. Through systematic preprocessing, model building, and evaluation, we were able to explore multiple classification models and identify the most effective solution.

Key Findings:

The Random Forest showed the best dependable model, demonstrating strong predictive accuracy and a best balance between correctly identifying defaults and avoiding false alarms which is shown below.

Highest ROC AUC (0.9633)
Strong F1 Score (0.8049)
Balanced precision (0.85) & recall (0.76)

XGBoost showed competitive results, slightly behind Random Forest, and can be considered for further tuning or ensemble techniques. Logistic Regression, though basic, demonstrated strong recall (0.82), meaning it rarely missed defaulters, but suffered from low precision (0.63), potentially flagging too many false positives. Decision Tree underperformed, indicating a need for deeper tree-based models or ensemble methods for such complex data.

Business Implications:

A robust credit default prediction model offers several advantages to banks and financial institutions. It helps minimize financial losses by identifying high-risk customers early. It improves customer experience by speeding up approvals for low-risk applicants. Such models also support regulatory compliance by being explainable and auditable. Additionally, they help reduce operational costs by automating parts of the decision-making process.

Recommendations

To further improve model performance and business adoption:

- **Feature Engineering:** Create features like trend indicators, rolling averages, or recent transaction history. Explore time-window aggregation for behavior-based features.
- **Model Optimization:** Perform more extensive hyperparameter tuning (e.g., using randomized search). Consider implementing ensemble techniques such as stacking by combining the strengths of high-performing models like Random Forest and XGBoost. This hybrid approach may enhance prediction stability and improve overall model performance.
- **Bias & Fairness Audits:** It is essential to examine whether the model's decisions are consistent across various demographic groups, such as different age brackets, genders, or income levels. This includes checking if certain groups are unfairly favored or penalized. Conducting fairness assessments can help confirm that the model behaves responsibly and does not introduce unintentional bias in credit evaluations.
- **Model Monitoring:** Set up a feedback loop to monitor predictions and update the model regularly. Use A/B testing to evaluate model performance against existing systems.
- **Interpretability:** We may utilize tools such as SHAP or LIME to advance the explainability of the model's predictions. Both tools can help in understanding how individual features contribute to the prediction outcomes. This level of transparency is highly significant when the results are used in customer-facing applications, as it provides clear justifications for each decision.

Final Recommendation

Based on current evaluation metrics, Random Forest is recommended for deployment as the primary model. However, continual monitoring, feedback integration, and further experimentation with advanced techniques will help refine predictions in real-world environments.

Appendix (Code and Output)

Appendix A (Data Preparation and Analysis)

Appendix A.1 Import Libraries and Data

Code:

```
import pandas as pd
import gc
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Appendix A.2 Load Data and Merge

Code:

```
train_iter_obj =
    pd.read_csv('/Users/anugrahrastogi21gmail.com/Downloads/amex-default-prediction/train_
data.csv', chunksize=7_000_0) # 0.5 million rows per chunk

for train_chunk in train_iter_obj:

    print(train_chunk.shape)

    # process or save each chunk

    train_chunk.to_csv('/Users/anugrahrastogi21gmail.com/Documents/USD_project/customer-
credit-default-prediction/01_data/train_data.csv', index=False)

break
```

```

train_df =
    pd.read_csv("/Users/anugrahrastogi21@gmail.com/Documents/USD_project/customer-credit-default-prediction/01_data/train_data.csv")

memory_bytes = train_df.memory_usage(deep=True).sum()

memory_mb = memory_bytes / (1024 * 1024)

print(memory_mb)

train_labels_df =
    pd.read_csv("/Users/anugrahrastogi21@gmail.com/Downloads/amex-default-prediction/train_labels.csv")

train_df_complete = pd.merge(train_df, train_labels_df, how="inner", on="customer_ID")

train_df_complete.shape, train_df.shape

```

Output:

```

117.51184463500977

((70000, 191), (70000, 190))

```

Appendix B EDA (Exploratory data analysis)

Appendix B.1 Description of the train dataset

Code:

```
train_df_complete.describe().T
```

Output:

	count	mean	std	min	25%	50%	75%	max
P_2	69427.0	0.655315	0.246852	-3.830190e-01	0.476650	0.694480	0.866688	1.009998
D_39	70000.0	0.154490	0.272714	8.701630e-07	0.004576	0.009121	0.236133	5.331360
B_1	70000.0	0.126764	0.215641	-1.956662e-01	0.008832	0.032111	0.129060	1.324053
B_2	69977.0	0.614745	0.402095	3.308280e-06	0.098566	0.814030	1.002162	1.010000
R_1	70000.0	0.080526	0.229780	2.880975e-07	0.002870	0.005756	0.008651	2.507711
...	...	...	...	...	...	...	...	...
D_142	11789.0	0.392237	0.243633	-1.394664e-02	0.188239	0.384605	0.569647	1.751388
D_143	68765.0	0.176145	0.376643	1.653580e-08	0.003019	0.006060	0.009040	1.010000
D_144	69490.0	0.054539	0.188182	4.053056e-07	0.002748	0.005501	0.008259	1.343284
D_145	68765.0	0.060399	0.196428	4.609489e-08	0.003030	0.006021	0.009040	4.282032
target	70000.0	0.252086	0.434213	0.000000e+00	0.000000	0.000000	1.000000	1.000000

187 rows × 8 columns

Appendix B.2 Count of numerical and categorical columns

Code:

```
train_df_complete['S_2'] = pd.to_datetime(train_df_complete['S_2'])

numerical_columns =
    train_df_complete.select_dtypes(include=['number']).columns.tolist()

categorical_columns = train_df_complete.select_dtypes(include=['object',
    'category']).columns.tolist()

print("number of numerical columns: ", len(numerical_columns), '\n',
      "number of categorical columns: ", len(categorical_columns))
```

Output:

number of numerical columns: 187

number of categorical columns: 3

Appendix B.3 Checking Null Values

Code:

```
null_value_df = pd.DataFrame({'column_name':train_df_complete.isnull().sum().index,  
                             'null_count':train_df_complete.isnull().sum().values})  
  
null_value_df["percent_missing"] = null_value_df["null_count"]*100 / len(train_df_complete)  
  
null_value_df.sort_values('percent_missing',  
                          ascending=False).to_csv("null_percentage_df.csv", index=False)
```

Appendix B.4 Removing Null Values

Code:

```
null_columns = null_value_df[null_value_df["percent_missing"] >  
                             90]["column_name"].to_list()  
  
print("count of null value columns to be dropped: ", len(null_columns))  
  
train_df_complete = train_df_complete.drop(null_columns, axis=1)  
  
print("shape of train dataset after dropping null valued columns: ",  
      train_df_complete.shape)
```

Appendix B.5 Removing Columns with >90% Null Values

Code:

```
null_columns = null_value_df[null_value_df["percent_missing"] >  
                             90]["column_name"].to_list()  
  
print("count of null value columns to be dropped: ", len(null_columns))  
  
train_df_complete = train_df_complete.drop(null_columns, axis=1)  
  
print("shape of train dataset after dropping null valued columns: ",  
      train_df_complete.shape)
```

Output:

count of null value columns to be dropped: 15

shape of train dataset after dropping null valued columns: (70000, 176)

Appendix B.6 Checking Zero Values

Code:

```
zero_value_df = pd.DataFrame({  
    'column_name': train_df_complete.columns,  
    'zero_count': (train_df_complete == 0).sum().values  
})  
  
# Add percentage of zero values  
zero_value_df['percent_zeros'] = (zero_value_df['zero_count'] * 100) /  
    len(train_df_complete)  
  
# Sort by highest percentage of zeros  
zero_value_df = zero_value_df.sort_values('percent_zeros', ascending=False)  
zero_value_df.to_csv("zero_percentage_df.csv", index=False)  
zero_value_df
```

Output:

	column_name	zero_count	percent_zeros
148	D_116	67703	96.718571
152	D_120	59542	85.060000
102	B_30	59400	84.857143
175	target	52354	74.791429
146	D_114	25380	36.257143
...	...	...	...
169	D_140	0	0.000000
171	D_142	0	0.000000
172	D_143	0	0.000000
173	D_144	0	0.000000
174	D_145	0	0.000000

176 rows × 3 columns

Appendix B.7 Removing Columns with >90% Null Values

Code:

```
zero_columns = zero_value_df[zero_value_df["percent_zeros"] >
90]["column_name"].to_list()

print("count of zero value columns to be dropped: ", len(zero_columns))

train_df_complete = train_df_complete.drop(zero_columns, axis=1)

print("shape of train dataset after dropping zero valued columns: ",
train_df_complete.shape)
```

Output:

```
count of zero value columns to be dropped: 1
shape of train dataset after dropping zero valued columns: (70000, 175)
```

Appendix B.8 Distribution of target variable

Code:

```
import matplotlib.pyplot as plt

import seaborn as sns

# Set seaborn style
sns.set(style="whitegrid")

# Calculate percentages
target_counts = train_df_complete['target'].value_counts(normalize=True) *
    100

# Plot
plt.figure(figsize=(6, 4))

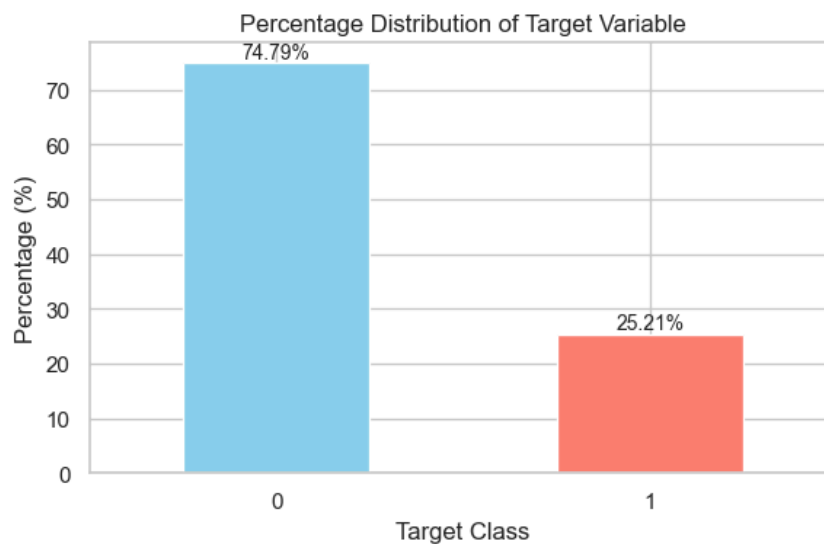
target_counts.plot(kind='bar', color=['skyblue', 'salmon'])

# Add labels and title
plt.xlabel('Target Class')
plt.ylabel('Percentage (%)')
plt.title('Percentage Distribution of Target Variable')
plt.xticks(rotation=0)

# Add percentage labels on bars
for i, value in enumerate(target_counts):
    plt.text(i, value + 1, f'{value:.2f}%', ha='center', fontsize=10)

plt.tight_layout()
plt.show()
```

Output:



Appendix C Data pre-processing

Appendix C.1 Box plot to observe the skewness and outliers

Code:

```
import pandas as pd

import matplotlib.pyplot as plt

from matplotlib.backends.backend_pdf import PdfPages

# Filter numerical columns

numerical_cols = train_df_complete.select_dtypes(include='number').columns

# Create PDF to save plots

with PdfPages('box_plots_numerical_features.pdf') as pdf:

    for col in numerical_cols:

        if col != 'target':

            plt.figure(figsize=(8, 4))

            plt.boxplot(train_df_complete[col].dropna(), vert=False)

            plt.title(f'Box Plot of {col}')

            plt.xlabel(col)

            plt.tight_layout()

            # Save each plot to the PDF

            pdf.savefig()

            plt.close()

    print("Box plots saved to 'box_plots_numerical_features.pdf'")
```

Appendix C.2 Remove Outliers

Code:

```
percentile25 = train_df_complete['P_2'].quantile(0.25)

percentile75 = train_df_complete['P_2'].quantile(0.75)

iqr = percentile75 - percentile25

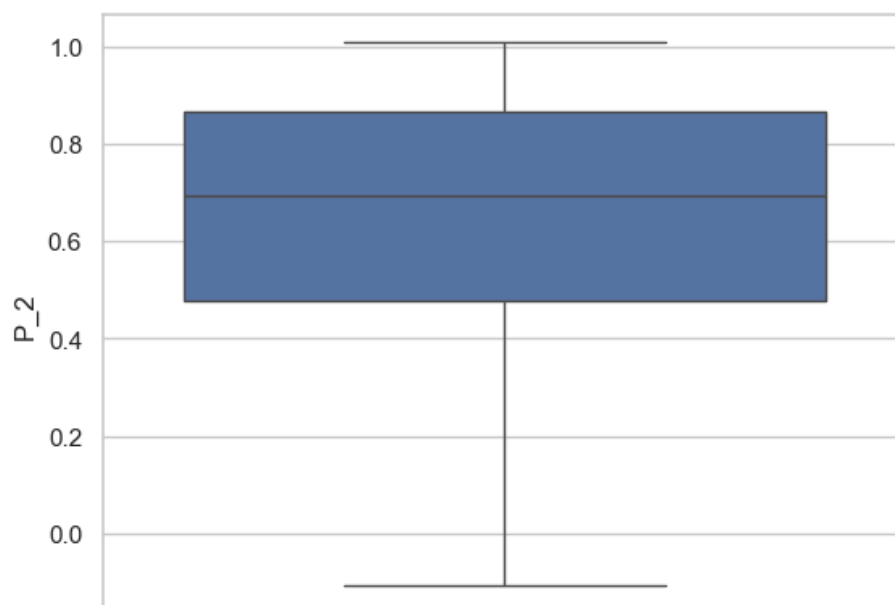
upper_limit = percentile75 + 1.5 * iqr

lower_limit = percentile25 - 1.5 * iqr

train_df_complete['P_2'] = np.where(train_df_complete['P_2'] >= upper_limit,
                                     upper_limit,
                                     np.where(train_df_complete['P_2'] <= lower_limit,
                                               lower_limit,
                                               train_df_complete['P_2']))

sns.boxplot(train_df_complete['P_2'])
```

Output:



Appendix C.3 Categorical variables distribution vs target

Code

```
for col in categorical_columns:

    if col != "customer_ID":

        plt.figure(figsize=(6, 4))

        sns.barplot(data=train_df_complete, x=col, y='target', estimator='mean',
                    order=train_df_complete[col].value_counts().index)

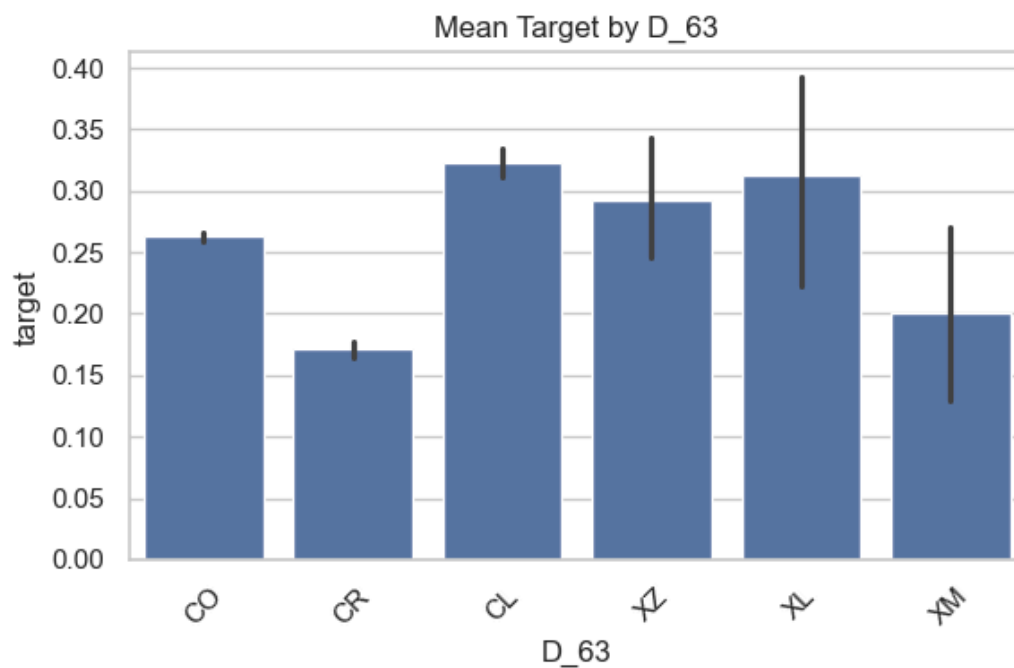
        plt.title(f'Mean Target by {col}')

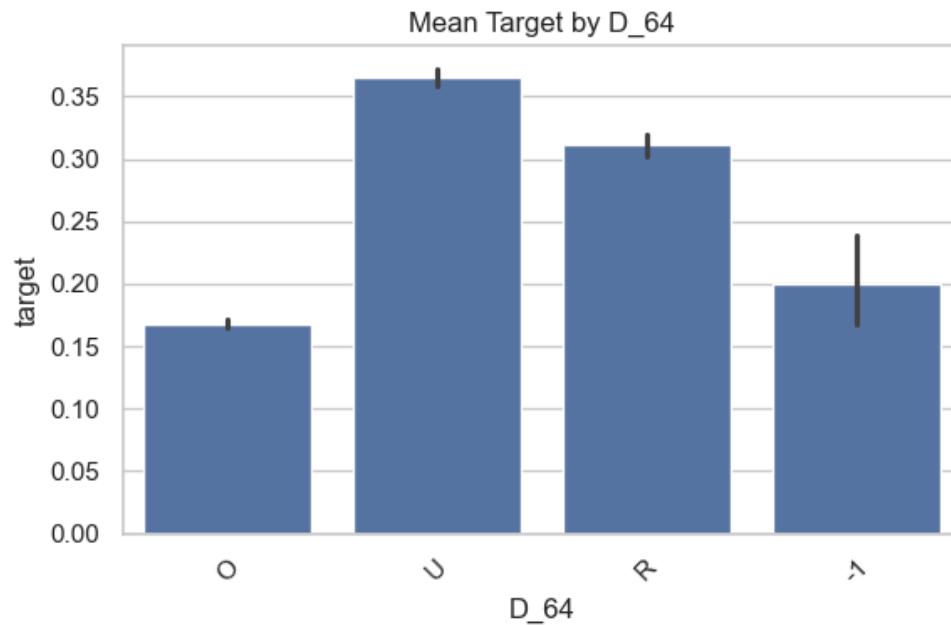
        plt.xticks(rotation=45)

        plt.tight_layout()

        plt.show()
```

Output:





Appendix D Feature selection technique (Information Value)

Appendix D.1 IV Calculation

Code:

```
def calculate_iv_all_features(df, target_col):
    """
    Calculate IV for all features in a DataFrame (excluding the target).

    Parameters:
    - df: pandas DataFrame
    - target_col: the name of the target column (binary: 0 = good, 1 = bad)
    - bins: number of bins for numeric features

    Returns:
    - DataFrame: sorted IV values for each feature
    """
    iv_dict = {}
    feature_cols = df.columns.drop([target_col, 'customer_ID'])

    for col in feature_cols:
        try:
            iv = calculate_iv(df, col, target_col)
            iv_dict[col] = iv
        except Exception as e:
            print(f"Skipped {col} due to error: {e}")
            continue

    iv_df = pd.DataFrame.from_dict(iv_dict, orient='index', columns=['IV'])
    iv_df = iv_df.sort_values(by='IV', ascending=False)
    return iv_df
```



```
# Example: Assuming your target column is named 'target'
iv_result = calculate_iv_all_features(train_df_complete, target_col='target').reset_index()
iv_result.to_csv("IV_result.csv", index=False)
```

```
def correlation(dataset, threshold, verbose=False):
    """
    Identify highly correlated features to drop based on a threshold.

    Parameters:
    - dataset (pd.DataFrame): The input DataFrame with numeric columns.
    - threshold (float): Correlation threshold above which to drop one of the variables.
    - verbose (bool): If True, prints each pair of correlated features.

    Returns:
    - col_corr (set): Set of column names to consider for dropping.
    """
    col_corr = set()
    corr_matrix = dataset.corr(numeric_only=True)

    for i in range(len(corr_matrix.columns)):
        for j in range(i):
            corr_value = abs(corr_matrix.iloc[i, j])
            if corr_value > threshold:
                colname = corr_matrix.columns[i]
                if verbose:
                    print(f"High correlation ({corr_value:.2f}) between: {colname} and {corr_matrix.columns[j]}")
                col_corr.add(colname)
    return col_corr
```

Appendix D.2 Removing features with IV < 0.02

Code:

```
columns_to_remove_weak_iv = iv_result[iv_result["IV"] < 0.02]["index"].to_list()
len(columns_to_remove_weak_iv)
```

Output:

38

Appendix D.3 Removing features with IV < 0.5

Code:

```
columns_to_remove_hyper_iv = iv_result[iv_result["IV"] > 0.5]["index"].to_list()
len(columns_to_remove_hyper_iv)
```

Output:

47

Appendix D.4 Removing features from the train dataset

Code:

```
columns_to_remove_weak_iv.extend(columns_to_remove_hyper_iv)
len(columns_to_remove_weak_iv)
```

Output:

85

Appendix D.5 Drop 85 features from IV method

Code:

```
train_df_complete.drop(columns_to_remove_weak_iv, axis=1, inplace=True)
train_df_complete.shape
```

Appendix D.6 Remove highly correlated variables using Pearson correlation

Code:

```
def correlation(dataset, threshold, verbose=False):
    """
    Identify highly correlated features to drop based on a threshold.

    Parameters:
    - dataset (pd.DataFrame): The input DataFrame with numeric columns.
    - threshold (float): Correlation threshold above which to drop one of the variables.
    - verbose (bool): If True, prints each pair of correlated features.

    Returns:
    - col_corr (set): Set of column names to consider for dropping.
    """
    col_corr = set()
    corr_matrix = dataset.corr(numeric_only=True)

    for i in range(len(corr_matrix.columns)):
        for j in range(i):
            corr_value = abs(corr_matrix.iloc[i, j])
            if corr_value > threshold:
                colname = corr_matrix.columns[i]
                if verbose:
                    print(f"High correlation ({corr_value:.2f}) between: {colname} and {corr_matrix.columns[j]}")
                col_corr.add(colname)
    return col_corr
```

Appendix D.7 Removing correlated features (threshold $\geq 75\%$)

Code:

```
high_corr_cols = correlation(train_df_complete, threshold=0.75, verbose=True)
train_df_complete_corr = train_df_complete.drop(columns=high_corr_cols)
print("Final train dataset shape: ", train_df_complete_corr.shape)
```

Output:

```
High correlation (0.78) between: R_4 and R_2
High correlation (0.92) between: B_13 and B_12
High correlation (0.80) between: S_15 and S_8
High correlation (0.79) between: R_8 and R_5
High correlation (0.78) between: S_16 and B_21
High correlation (0.77) between: D_84 and D_53
High correlation (1.00) between: D_104 and D_103
High correlation (0.85) between: D_107 and D_103
High correlation (0.84) between: D_107 and D_104
High correlation (0.82) between: D_118 and D_115
High correlation (0.82) between: D_119 and D_115
High correlation (1.00) between: D_119 and D_118
High correlation (0.75) between: D_129 and D_128
High correlation (0.90) between: D_131 and D_79
High correlation (0.92) between: D_132 and D_131
High correlation (1.00) between: D_141 and D_139
High correlation (0.80) between: D_142 and D_141
High correlation (1.00) between: D_143 and D_139
High correlation (1.00) between: D_143 and D_141
Final train dataset shape: (70000, 74)
```

Appendix D.8 Perform one-hot encoding for 2 categorical features

Code:

```
cat_cols = ["D_64", "D_63"]
train_df_complete_corr_encl = pd.get_dummies(train_df_complete_corr, columns=cat_cols)
```

Appendix D.9 Saving Processed and Final Train and Test Data Sets

Code:

```
train_df_complete_corr_encl.to_csv("/Users/anugrahrastogi21@gmail.com/Documents/USD_project/customer-credit-default-prediction/01_data/processed_train_data.csv",
                                   index=False)
```

Note: Since the test dataset was large (33GB) for GitHub, we compressed it to 33k records for repository inclusion.

Appendix E Model Selection and Evaluation

Appendix E.1 Load Data

Code:

```
# Step 1: Import Libraries and Load Data

import os
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder, RobustScaler
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, roc_auc_score, f1_score

from xgboost import XGBClassifier

# Setup
warnings.filterwarnings('ignore')
%matplotlib inline

# File paths

train_file = "processed_train_data.csv"

test_file = "processed_test_data.csv"

# Validate file presence

if not os.path.exists(train_file) or not os.path.exists(test_file):

    raise FileNotFoundError(f"Missing file(s): {train_file if not os.path.exists(train_file)
    else ''} {test_file if not os.path.exists(test_file) else ''}")

# Load CSVs

train_df = pd.read_csv(train_file)

test_df = pd.read_csv(test_file)

# Shape report

print(f"Train Dataset: {train_df.shape}")

print(f"Test Dataset: {test_df.shape}")
```

Output:

Train Dataset: (7000, 78)

Test Dataset: (7000, 76)

Appendix E.2 Preprocess Data (Data Cleaning)

Code:

```
# Step 2: Clean and preprocess the data

# Define target column
target_feature = 'target'
if target_feature not in train_df.columns:
    raise ValueError(f"Target column '{target_feature}' not found in training data")

# Separate features and target
input_train_data = train_df.drop(columns=[target_feature])
output_train_data = train_df[target_feature]
input_test_data = test_df.drop(columns=[target_feature], errors='ignore')

# Align train and test columns
shared_features = input_train_data.columns.intersection(input_test_data.columns)
input_train_data = input_train_data[shared_features]
input_test_data = input_test_data[shared_features]
print(f"Aligned columns: {len(shared_features)}")

# Handle missing values
num_features = input_train_data.select_dtypes(include=[np.number]).columns
for col in num_features:
    fill_value = input_train_data[col].median()
    input_train_data[col].fillna(fill_value, inplace=True)
    input_test_data[col].fillna(fill_value, inplace=True)
```

```

# Encode categorical columns
encoder_dict = {}
cat_features = input_train_data.select_dtypes(include=['object', 'bool']).columns
for col in cat_features:
    encoder = LabelEncoder()
    combined_values = pd.concat([input_train_data[col], input_test_data[col]], axis=0).astype(str).unique()
    encoder.fit(combined_values)
    input_train_data[col] = encoder.transform(input_train_data[col].astype(str))
    input_test_data[col] = encoder.transform(input_test_data[col].astype(str))
    encoder_dict[col] = encoder

# Split training data into train and validation sets
X_train_set, X_val_set, y_train_set, y_val_set = train_test_split(
    input_train_data, output_train_data, test_size=0.2, random_state=42, stratify=output_train_data
)

# Scale features
robust_scaler = RobustScaler()
X_train_scaled = robust_scaler.fit_transform(X_train_set)
X_val_scaled = robust_scaler.transform(X_val_set)
X_test_input = robust_scaler.transform(input_test_data)

print(f"Training set: {X_train_set.shape}, Validation set: {X_val_set.shape}, Test set: {input_test_data.shape}")

```

Output:

Aligned columns: 76

Training set: (5600, 76), Validation set: (1400, 76), Test set: (7000, 76)

Appendix E.3 Train Models with Hyperparameter Tuning

Code:

```

# Initialize models and parameter grids
model_dict = {
    'RandomForest': {
        'model': RandomForestClassifier(random_state=42, n_jobs=-1),
        'param_grid': {
            'n_estimators': [50, 100],
            'max_depth': [10, 20, None],
            'min_samples_split': [2, 5]
        }
    },
    'DecisionTree': {
        'model': DecisionTreeClassifier(random_state=42),
        'param_grid': {
            'max_depth': [5, 10, 15],
            'min_samples_split': [2, 5],
            'min_samples_leaf': [1, 2]
        }
    },
    'XGBoost': {
        'model': XGBClassifier(random_state=42, n_jobs=-1, eval_metric='logloss'),
        'param_grid': {
            'n_estimators': [50, 100],
            'max_depth': [3, 6],
            'learning_rate': [0.1, 0.01]
        }
    },
    'LogisticRegression': {
        'model': LogisticRegression(random_state=42, max_iter=10000, class_weight='balanced'),
        'param_grid': {
            'C': [0.1, 1.0, 10.0],
            'solver': ['lbfgs', 'liblinear']
        }
    }
}

```

```

# Train and tune models
best_model_dict = {}
for model_name, model_conf in model_dict.items():
    print(f"\nTuning {model_name}...")
    grid_search = GridSearchCV(
        model_conf['model'], model_conf['param_grid'], cv=5, scoring='roc_auc', n_jobs=-1
    )
    grid_search.fit(X_train_scaled, y_train_set)
    best_model_dict[model_name] = grid_search.best_estimator_
    print(f"Best parameters: {grid_search.best_params_}")
    print(f"Best ROC-AUC score from cross-validation: {grid_search.best_score_:.4f}")

```

Output:

Tuning RandomForest

- Best parameters: {'max_depth': None, 'min_samples_split': 2, 'n_estimators': 100}
- Best ROC-AUC score from cross-validation: 0.9559

Tuning DecisionTree

- Best parameters: {'max_depth': 10, 'min_samples_leaf': 2, 'min_samples_split': 5}
- Best ROC-AUC score from cross-validation: 0.8708

Tuning XGBoost

- Best parameters: {'learning_rate': 0.1, 'max_depth': 6, 'n_estimators': 100}
- Best ROC-AUC score from cross-validation: 0.9536

Tuning LogisticRegression

- Best parameters: {'C': 0.1, 'solver': 'liblinear'}
- Best ROC-AUC score from cross-validation: 0.9074

Appendix E.4: Model Evaluation

All models were assessed using the parameters ROC-AUC, F1-score, and a detailed classification report, as demonstrated in the code below.

Code:

```
# Evaluate each model
validation_results = {}
for model_label, tuned_model in best_model_dict.items():
    print(f"\n{model_label} Validation Results:")
    y_valid_pred = tuned_model.predict(X_val_scaled)
    y_valid_proba = tuned_model.predict_proba(X_val_scaled)[:, 1]

    validation_results[model_label] = {
        'roc_auc': roc_auc_score(y_val_set, y_valid_proba),
        'f1_score': f1_score(y_val_set, y_valid_pred),
        'y_val_pred': y_valid_pred,
        'y_val_proba': y_valid_proba
    }

print(classification_report(y_val_set, y_valid_pred))
print(f"ROC AUC: {validation_results[model_label]['roc_auc']:.4f}")
print(f"F1 Score: {validation_results[model_label]['f1_score']:.4f}")
```


Output:

Here's the detailed report of precision, recall, F1-score, and support.

RandomForest Validation Results:
precision recall f1-score support

0	0.94	0.97	0.96	1066
1	0.91	0.80	0.85	334

accuracy			0.93	1400
macro avg	0.92	0.89	0.90	1400
weighted avg	0.93	0.93	0.93	1400

ROC AUC: 0.9777
F1 Score: 0.8508

DecisionTree Validation Results:
precision recall f1-score support

0	0.89	0.94	0.91	1066
1	0.75	0.61	0.68	334

accuracy			0.86	1400
macro avg	0.82	0.78	0.79	1400
weighted avg	0.85	0.86	0.85	1400

ROC AUC: 0.8344
F1 Score: 0.6766

XGBoost Validation Results:
precision recall f1-score support

0	0.96	0.99	0.97	1066
1	0.96	0.86	0.91	334

accuracy			0.96	1400
macro avg	0.96	0.92	0.94	1400
weighted avg	0.96	0.96	0.96	1400

ROC AUC: 0.9916
F1 Score: 0.9082

LogisticRegression Validation Results:
precision recall f1-score support

0	0.95	0.83	0.89	1066
1	0.62	0.85	0.71	334

accuracy			0.84	1400
macro avg	0.78	0.84	0.80	1400
weighted avg	0.87	0.84	0.85	1400

ROC AUC: 0.9158
F1 Score: 0.7137

Model Comparison Table

Model	Accuracy	ROC AUC	F1 Score (Class 1)	Precision (Class 1)	Recall (Class 1)
Random Forest	0.91	0.9633	0.8049	0.85	0.76
XGBoost	0.90	0.9576	0.7906	0.82	0.76
Decision Tree	0.85	0.8808	0.6855	0.75	0.63
Logistic Regression	0.83	0.9057	0.7134	0.63	0.82

Appendix E.5: Model Performance Comparison

To compare the effectiveness of all trained models, we evaluated them using the validation dataset based on metrics: Accuracy, ROC AUC, F1-Score, Precision score, and Recall score.

A grouped bar chart was generated to visually compare the ROC AUC and F1 scores across all models, using the code provided below.

Code:

```

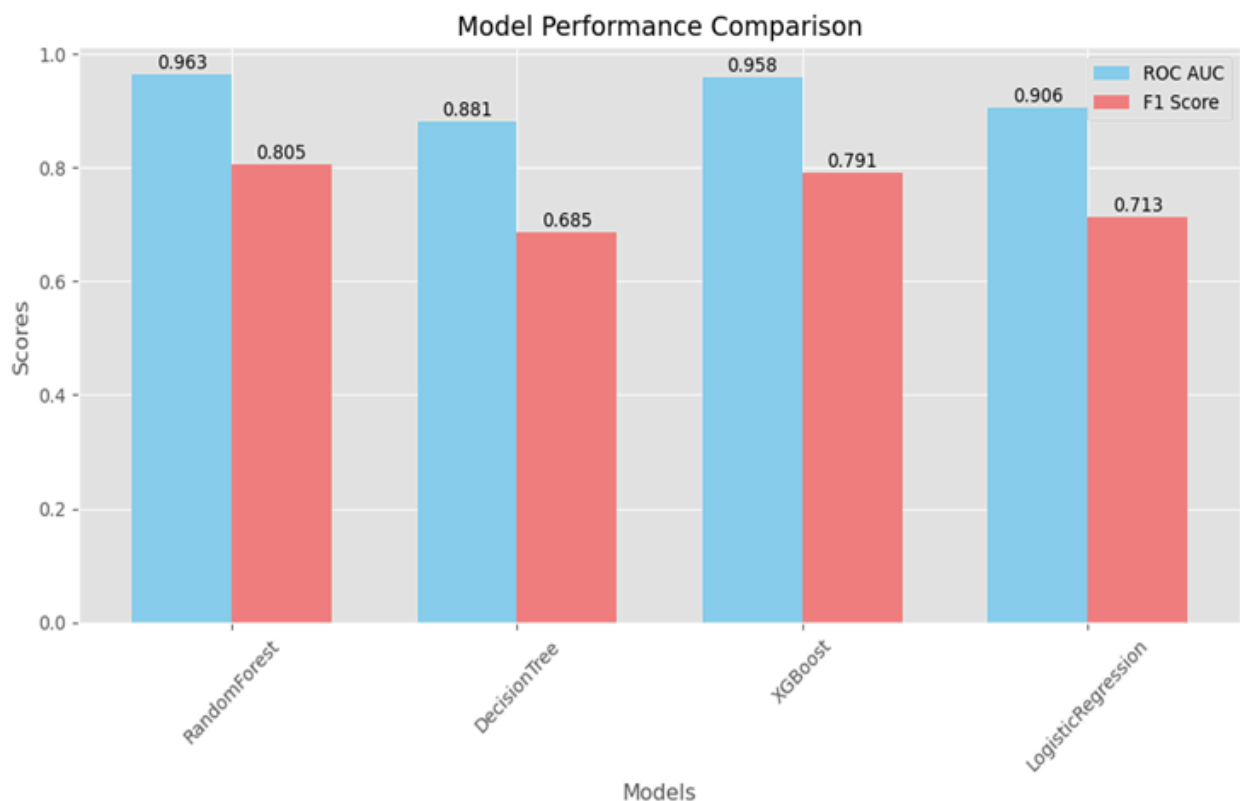
# Collect metrics
model_names = list(validation_results.keys())
roc_auc_scores = [validation_results[name]['roc_auc'] for name in model_names]
f1_scores = [validation_results[name]['f1_score'] for name in model_names]

# Plot comparison
fig, ax = plt.subplots(figsize=(10, 6))
x = np.arange(len(model_names))
width = 0.35
ax.bar(x - width/2, roc_auc_scores, width, label='ROC AUC', color='skyblue')
ax.bar(x + width/2, f1_scores, width, label='F1 Score', color='lightcoral')
ax.set_title('Model Performance Comparison')
ax.set_ylabel('Scores')
ax.set_xlabel('Models')
ax.set_xticks(x)
ax.set_xticklabels(model_names, rotation=45)

ax.legend()
for i, (roc, f1) in enumerate(zip(roc_auc_scores, f1_scores)):
    ax.text(i - width/2, roc + 0.01, f'{roc:.3f}', ha='center')
    ax.text(i + width/2, f1 + 0.01, f'{f1:.3f}', ha='center')
plt.tight_layout()
plt.show()

```

Output:



Appendix E.6: Make Final Predictions

Using the ROC-AUC evaluation, choose the best model and run predictions on the unseen test set with the following snippet.

Code:

```
# Select best model based on ROC-AUC
best_model_name = max(validation_results, key=lambda x: validation_results[x]['roc_auc'])
best_model = best_models[best_model_name]
print(f"Best model: {best_model_name} (ROC AUC: {validation_results[best_model_name]['roc_auc']:.4f})")

# Make predictions
predictions = best_model.predict(X_test_scaled)
probabilities = best_model.predict_proba(X_test_scaled)[: , 1]
print(f"Generated {len(predictions)} predictions")
print(f"Default rate: {(predictions == 1).mean():.4f}")

# Save predictions
output = pd.DataFrame({'Prediction': predictions, 'Probability': probabilities})
output.to_csv('test_predictions.csv', index=False)
print("Predictions saved to 'test_predictions.csv'")
```

Output:

Best model: RandomForest (ROC AUC: 0.9633)

Generated 33000 predictions

Default rate: 0.2061

Predictions saved to 'test_predictions.csv'

Appendix E.7: Visualizations

To support the model evaluation and comparison, we generated the following visualizations.

- 1. ROC Curve:** It shows both True and False Positive Rates. It shows how they vary as the classification threshold changes, highlighting the model's performance trade-offs.
- 2. Precision - Recall Curve:** It is very useful in imbalanced data sets as it considers only the minority class (defaults in our case).

The following code is used to generate ROC Curve and Precision-Recall Curve.

Code:

```

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import roc_curve, precision_recall_curve, auc
import numpy as np
import pandas as pd

# Set plotting style
plt.style.use('ggplot')

# Recalculate ROC and PR curves using consistent y_val length
fig = plt.figure(figsize=(15, 12))

```

```

# 1. ROC Curve
plt.subplot(2, 2, 1)
for model_name, model in best_models.items():
    y_val_proba = model.predict_proba(X_val_scaled)[: , 1]

    # Slice y_val to match length if needed
    if len(y_val) != len(y_val_proba):
        min_len = min(len(y_val), len(y_val_proba))
        y_val_plot = y_val[:min_len]
        y_val_proba = y_val_proba[:min_len]
    else:
        y_val_plot = y_val

    fpr, tpr, _ = roc_curve(y_val_plot, y_val_proba)
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f'{model_name} (AUC = {roc_auc:.3f})')

plt.plot([0, 1], [0, 1], 'k--', label='Random')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves')
plt.legend(loc='lower right')
plt.grid(True)

```

```
# 2. Precision-Recall Curve
plt.subplot(2, 2, 2)
for model_name, model in best_models.items():
    y_val_proba = model.predict_proba(X_val_scaled)[: , 1]

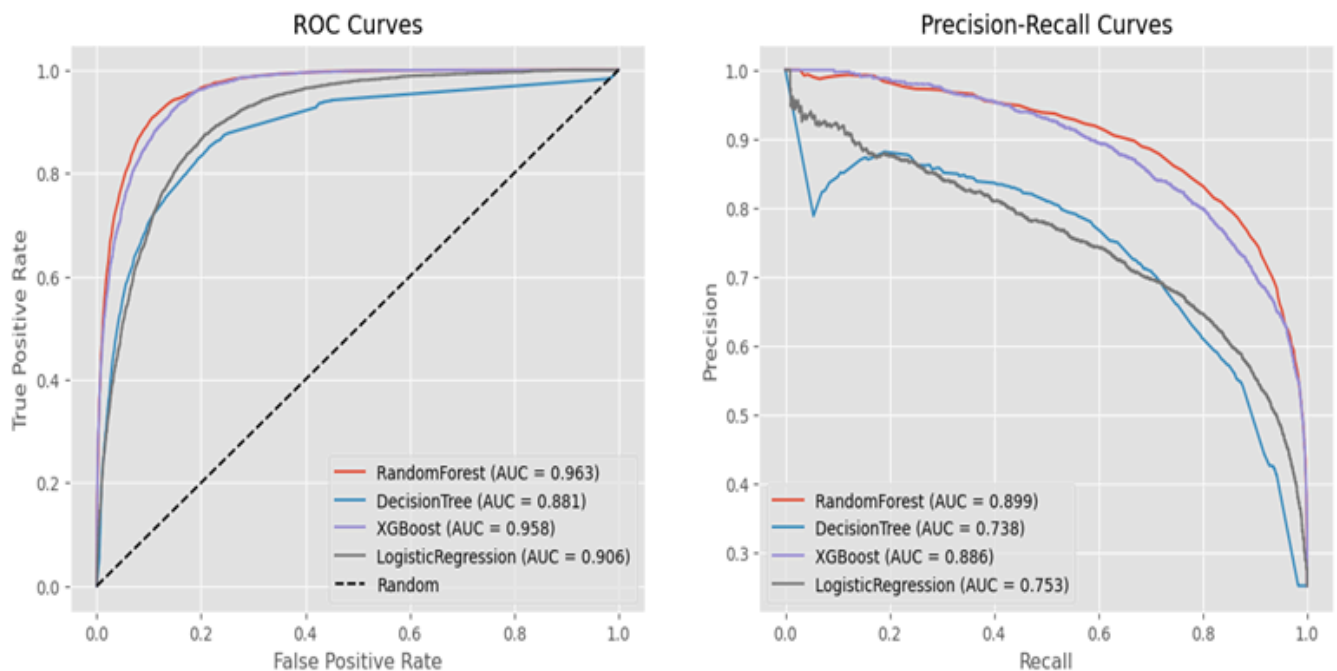
    if len(y_val) != len(y_val_proba):
        min_len = min(len(y_val), len(y_val_proba))
        y_val_plot = y_val[:min_len]
        y_val_proba = y_val_proba[:min_len]
    else:
        y_val_plot = y_val

    precision, recall, _ = precision_recall_curve(y_val_plot, y_val_proba)
    pr_auc = auc(recall, precision)
    plt.plot(recall, precision, label=f'{model_name} (AUC = {pr_auc:.3f})')

plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curves')
plt.legend(loc='lower left')
plt.grid(True)

plt.tight_layout()
plt.show()
```

Output:



These visual insights help in making data-backed decisions and further validate the Random Forest model as the final choice based on stable and well-balanced performance across key evaluation metrics.

References

1. Kaggle. (2023). *Customer Credit Default Prediction Dataset*. Retrieved from <https://www.kaggle.com/datasets>
2. Pedregosa, F., et al. (2011). *GridSearchCV – Scikit-learn 1.4.2 documentation*. Scikit-learn.
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
3. Pedregosa, F., et al. (2011). *RobustScaler – Scikit-learn 1.4.2 documentation*. Scikit-learn.
<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.RobustScaler.html>
4. Pedregosa, F., et al. (2011). *OneHotEncoder – Scikit-learn 1.4.2 documentation*. Scikit-learn.
<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html>
5. Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied Logistic Regression* (3rd ed.). Wiley.
6. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
<https://doi.org/10.1023/A:1010933404324>
7. Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81–106.
<https://doi.org/10.1007/BF00116251>
8. Chen, T., & Guestrin, C. (2016, August). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM.
<https://doi.org/10.1145/2939672.2939785>